

Tarea_2_2_validacion_hiperparametros

June 6, 2026

1 Tarea 2.2 - Validacion y ajuste de hiperparametros

Este cuaderno documenta de forma tecnica **como** se realizo la validacion y ajuste de hiperparametros, y **por que** la Tarea 2.2 se considera concluida.

1.1 Alcance y criterio de cierre

Criterio del Product Backlog para la Tarea 2.2: - Aplicar tecnicas de validacion sobre un conjunto de evaluacion. - Ajustar hiperparametros para estimar el rendimiento y evitar sobreajuste.

Ademas, esta tarea mantiene la misma matriz de entrada usada en Tarea 2.1: - 1433 caracteristicas de texto del dataset Cora. - 4 caracteristicas relacionales conservadas tras Tarea 1.5. - Total: 1437 caracteristicas.

```
[1]: from pathlib import Path
import json
import re
import ast
import pandas as pd

def resolve_base_dir() -> Path:
    cwd = Path.cwd()
    if (cwd / 'artifacts').exists() and (cwd / 'Product Backlog.md').exists():
        return cwd
    if (cwd.parent / 'artifacts').exists() and (cwd.parent / 'Product Backlog.
↪md').exists():
        return cwd.parent
    raise FileNotFoundError('No se pudo resolver BASE_DIR desde el directorio_
↪actual.')
```

```
BASE_DIR = resolve_base_dir()
ART_DIR = BASE_DIR / 'artifacts' / 'tarea_2_1' / '20260527_124833'
CV_PATH = ART_DIR / 'resumen_cv.csv'
TEST_PATH = ART_DIR / 'resumen_test.csv'
META_PATH = ART_DIR / 'metadata.json'
BACKLOG_PATH = BASE_DIR / 'Product Backlog.md'

for p in [CV_PATH, TEST_PATH, META_PATH, BACKLOG_PATH]:
```

```

if not p.exists():
    raise FileNotFoundError(f'No existe: {p}')

print('BASE_DIR:', BASE_DIR)
print('Artefactos cargados desde:', ART_DIR)

```

BASE_DIR: c:\Users\juana\aprendizaje-automatico-relacional
Artefactos cargados desde: c:\Users\juana\aprendizaje-automatico-relacional\artifacts\tarea_2_1\20260527_124833

```

[2]: resumen_cv = pd.read_csv(CV_PATH)
resumen_test = pd.read_csv(TEST_PATH)
metadata = json.loads(META_PATH.read_text(encoding='utf-8'))

backlog_text = BACKLOG_PATH.read_text(encoding='utf-8')
lineas_22 = [ln.strip() for ln in backlog_text.splitlines() if 'Tarea 2.2' in ln]

print('Linea(s) de backlog para Tarea 2.2:')
for ln in lineas_22:
    print('-', ln)

print('\nModelo ganador reportado en metadata:', metadata.get('modelo_ganador'))
print('Metricas del ganador:', metadata.get('metricas_modelo_ganador'))

```

Linea(s) de backlog para Tarea 2.2:

- - **Tarea 2.2: Validación y ajuste de hiper-parámetros.** Aplicar técnicas de validación sobre un conjunto de evaluación para estimar el rendimiento de los modelos y evitar el sobreajuste (`_overfitting_`).

Modelo ganador reportado en metadata: ArbolDecision

Metricas del ganador: {'modelo': 'ArbolDecision', 'accuracy': 0.7712177121771218, 'precision_macro': 0.7627572785434565, 'recall_macro': 0.7602812499550776, 'f1_macro': 0.7587957177301675}

```

[6]: def parse_best_params(x):

    if isinstance(x, dict):

        return x

    s = str(x)

    # Limpia patrones como np.float64(1e-08) sin romper tuplas ni otros
    # paréntesis.

    s = re.sub(r"np\.float64\(((\[^\]]*\))\)", r"\1", s)

```

```

try:

    return ast.literal_eval(s)

except Exception:

    return s

cv = resumen_cv.copy()

cv['best_params'] = cv['best_params'].apply(parse_best_params)

 analisis = cv[['modelo', 'best_cv_f1_macro']].merge(

    resumen_test[['modelo', 'f1_macro', 'accuracy', 'precision_macro',
    ↪ 'recall_macro']],

    on='modelo',

    how='inner'

)

 analisis = analisis.rename(columns={'f1_macro': 'test_f1_macro'})

 analisis['gap_cv_test'] = analisis['best_cv_f1_macro'] -
    ↪ analisis['test_f1_macro']

 analisis = analisis.sort_values('best_cv_f1_macro', ascending=False).
    ↪ reset_index(drop=True)

print('Tabla tecnica de validacion y generalizacion (CV vs Test):')

display(analisis.style.format({

    'best_cv_f1_macro': '{:.4f}',

    'test_f1_macro': '{:.4f}',

    'gap_cv_test': '{:+.4f}',

```

```

        'accuracy': '{:.4f}',

        'precision_macro': '{:.4f}',

        'recall_macro': '{:.4f}',

    )))

```

Tabla tecnica de validacion y generalizacion (CV vs Test):

<pandas.io.formats.style.Styler at 0x20668751590>

```

[7]: print('Mejores hiperparametros por modelo:')
      for _, row in cv.sort_values('best_cv_f1_macro', ascending=False).iterrows():
          print(f"- {row['modelo']}: CV F1-macro={row['best_cv_f1_macro']:.4f} |
          ↪params={row['best_params']}")

```

Mejores hiperparametros por modelo:

```

- ArbolDecision: CV F1-macro=0.7477 | params={'clf__criterion': 'gini',
'clf__max_depth': 20, 'clf__min_samples_split': 5}
- MLP: CV F1-macro=0.7057 | params={'clf__alpha': 0.001,
'clf__hidden_layer_sizes': (64,), 'clf__learning_rate_init': 0.01}
- NaiveBayes: CV F1-macro=0.4081 | params={'clf__var_smoothing': 1e-08}
- kNN: CV F1-macro=0.3897 | params={'clf__n_neighbors': 3, 'clf__p': 2,
'clf__weights': 'distance'}

```

```

[8]: # Criterios de cierre documentados de forma trazable
top_model = analisis.iloc[0]
umbral_gap = 0.03

checklist = pd.DataFrame([
    {
        'criterio': 'Se aplico validacion tecnica durante el ajuste',
        'evidencia': 'GridSearchCV con StratifiedKfold (5 folds) en Tarea 2.1 y
        ↪resumen_cv.csv',
        'estado': 'Cumplido'
    },
    {
        'criterio': 'Se ajustaron hiperparametros en al menos 3 modelos',
        'evidencia': f"Modelos evaluados: {'', '.join(cv['modelo'].tolist())}",
        'estado': 'Cumplido' if cv['modelo'].nunique() >= 3 else 'Pendiente'
    },
    {
        'criterio': 'Se estimo capacidad de generalizacion para controlar
        ↪sobreajuste',
        'evidencia': f"Gap CV-Test del mejor modelo ({top_model['modelo']}):
        ↪{top_model['gap_cv_test']:.4f}",
    }
])

```

```

        'estado': 'Cumplido' if abs(top_model['gap_cv_test']) <= umbral_gap
    ↪else 'Revisar'
    },
    {
        'criterio': 'Se selecciono configuracion ganadora con metrica macro',
        'evidencia': f"Modelo ganador: {metadata.get('modelo_ganador')} | Test_
    ↪F1-macro: {metadata.get('metricas_modelo_ganador', {}).get('f1_macro', 'NA')}:
    ↪.4f}",
        'estado': 'Cumplido'
    },
    {
        'criterio': 'La matriz X de trabajo se mantiene respecto a Tarea 2.1_
    ↪(1437 features)',
        'evidencia': '1433 texto + 4 relacionales conservadas de Tarea 1.5',
        'estado': 'Cumplido'
    },
]
])

display(checklist)

```

	criterio \	evidencia	estado
0	Se aplico validacion tecnica durante el ajuste		
1	Se ajustaron hiperparametros en al menos 3 mod...		
2	Se estimo capacidad de generalizacion para con...		
3	Se selecciono configuracion ganadora con metri...		
4	La matriz X de trabajo se mantiene respecto a ...		
0	GridSearchCV con StratifiedKFold (5 folds) en ...		Cumplido
1	Modelos evaluados: ArbolDecision, MLP, NaiveBa...		Cumplido
2	Gap CV-Test del mejor modelo (ArbolDecision): ...		Cumplido
3	Modelo ganador: ArbolDecision Test F1-macro:...		Cumplido
4	1433 texto + 4 relacionales conservadas de Tar...		Cumplido

1.2 Conclusión tecnica de cierre

La Tarea 2.2 se considera concluida porque: 1. Se aplico validacion robusta para seleccionar hiperparametros (validacion cruzada estratificada). 2. Se comparo rendimiento en CV y test, aportando evidencia de generalizacion y control de sobreajuste. 3. Se obtuvo una configuracion ganadora documentada en artefactos reproducibles. 4. Se mantiene trazabilidad metodologica con el backlog y con la matriz X consolidada en Tarea 2.1.

Con esto, el objetivo de validacion y ajuste de hiperparametros queda cumplido y justificado para avanzar a la seleccion final del modelo (Tarea 2.3).